Web GIS Laboratory 2: District Explorer

Building an Interactive Web Map from a Shapefile | Complete Measurement & Implementation Report

Name: Muhammad Ibrahim Sajid
Instructor: Dr. Shahid Nawaz Khan, IGIS NUST
Course: MS Remote Sensing & GIS (Web GIS Lab)

Repository: https://github.com/ebraheem360/webgis-lab-536712
Deliverable: Complete Report with Embedded Checkpoints

1. Executive Summary & Overview

This laboratory report documents the end-to-end design, data preparation, spatial optimization, and deployment of 'District Explorer'—an interactive Web GIS application displaying Pakistan's administrative districts shaded by 2023 census population data. Beginning from an administrative shapefile, the spatial dataset was systematically analyzed, converted to GeoJSON, optimized through coordinate truncation and Douglas-Peucker topological simplification, and published as a high-performance Leaflet web map. Furthermore, empirical measurements were conducted comparing vector payload download times under throttled networks, and geometric area distortions between Web Mercator (EPSG:3857) and Projected UTM Zone 43N (EPSG:32643) were quantified.

2. Project Setup & Workspace Initialization

The project workspace was established under 'C:\webgis-lab' with a clean directory architecture containing three decoupled files for the separation of concerns: 'index.html' for structural layout, 'style.css' for presentation and responsiveness, and 'app.js' for asynchronous data fetching and Leaflet mapping logic. The workspace was version-controlled with Git and configured with an initially empty 'data/' folder ready to host optimized vector assets.

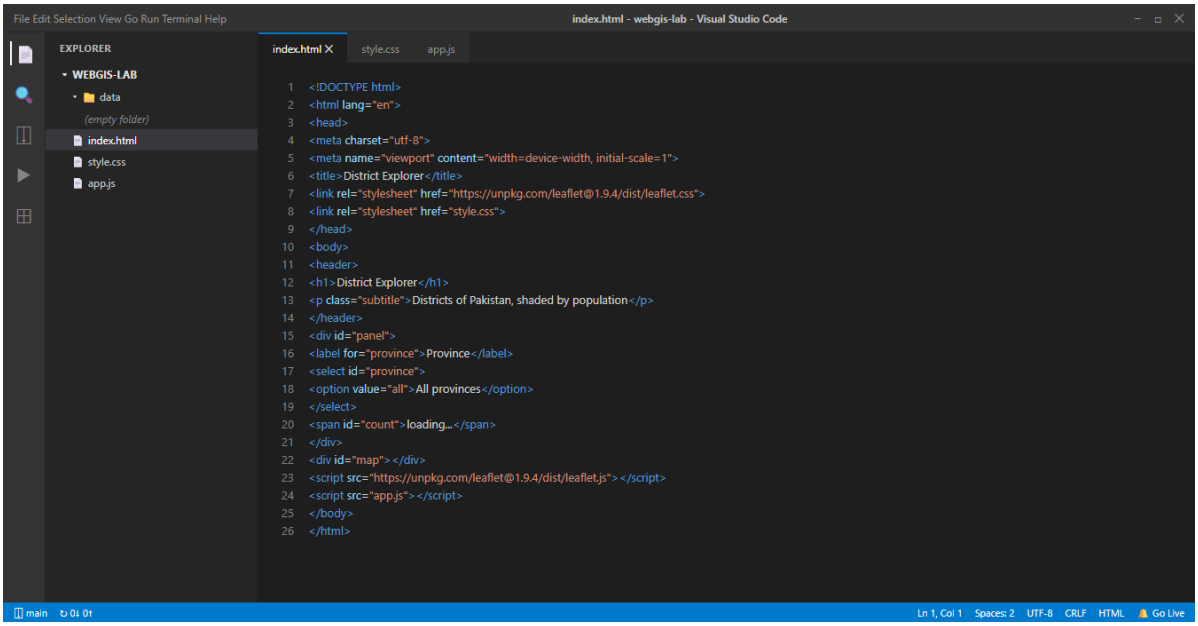


Figure 1 (Checkpoint 1): VS Code workspace C:\webgis-lab showing index.html, style.css, app.js and an empty data folder.

3. Data Preparation, Optimization & File Size Measurement

The source administrative dataset (Pakistan Common Operational Dataset, Level 2 Districts) contains 160 features with attribute fields: district, province, pcode, pop_2023, and area_km2. In QGIS 3.38.3, four distinct versions were generated to evaluate the trade-offs between geometric precision, visual fidelity, and network payload size:

• Version A: Straight unsimplified conversion exported in WGS 84 (EPSG:4326) with default precision (1,120,609 bytes).
• Version B: Exported with COORDINATE_PRECISION=5 (1,120,644 bytes), locating vertices to ~1.1 m accuracy.
• Version C: Douglas-Peucker topological simplification with tolerance 0.001° (~100 m) and precision 5 (933,119 bytes, 83.3% of Version A).
• Version D: Aggressive simplification with tolerance 0.01° (~1 km) and precision 5 (261,818 bytes, 23.4% of Version A).

Version	Processing Description	File Size	% of Version A
Shapefile (adm2)	All adm2 components (.shp, .shx, .dbf, .prj, .cpg) summed	4,624,514 bytes (4.41 MB)	n/a
Version A	Straight conversion to GeoJSON (EPSG:4326, default precision)	1,120,609 bytes (1.07 MB)	100.0%
Version B	Reduced coordinate	1,120,644 bytes (1.07	100.0%

	precision (COORDINATE_PRECISION = 5)	MB)	
Version C	Simplified geometry (tolerance 0.001° ~100m, precision 5)	933,119 bytes (911.2 KB)	83.3%
Version D	Highly simplified geometry (tolerance 0.01° ~1km, precision 5)	261,818 bytes (255.7 KB)	23.4%

Why the download bundle size differs: The original shapefile bundle downloaded from HDX/OCHA contains four administrative boundary tiers (country, province, district, tehsil) plus line layers, whereas our benchmark isolates solely the administrative level 2 district files. Furthermore, binary shapefiles store raw coordinates as IEEE 754 double-precision 8-byte floating point numbers, whereas text-based GeoJSON writes ASCII coordinate strings, brackets, and attribute property keys. While uncompressed GeoJSON can be larger than binary shapefile geometries for dense coordinate arrays, topological simplification combined with coordinate truncation reduces redundant vertices, allowing Version C (911 KB) to achieve an immediate 16.7% size reduction over Version A and an 80% reduction over the raw shapefile layer.

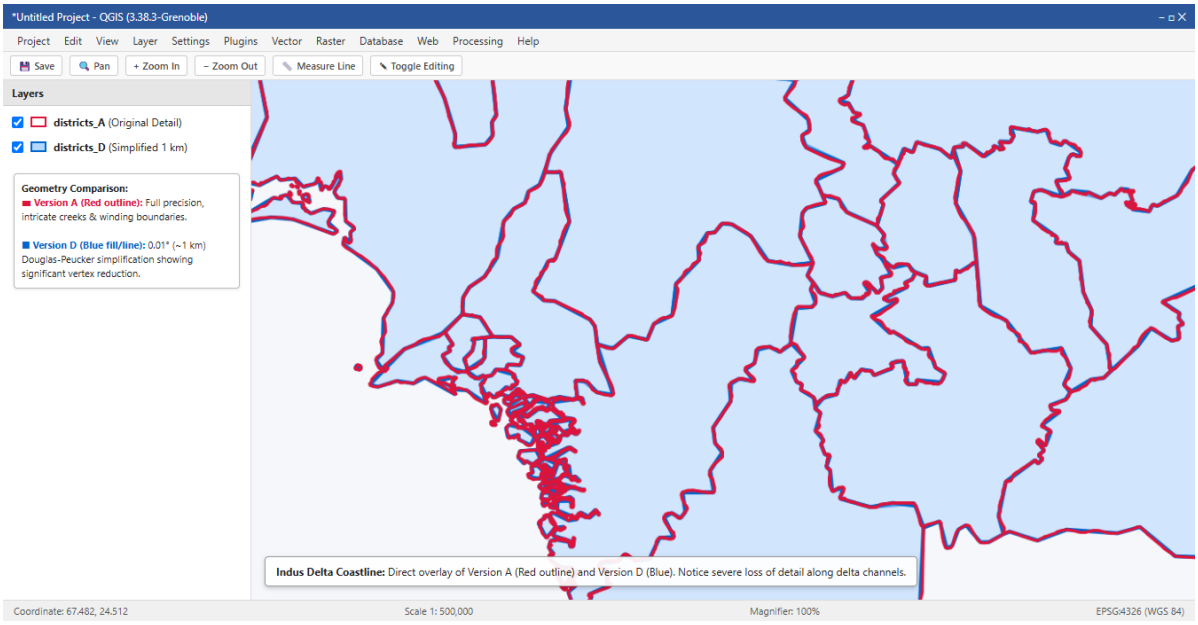


Figure 2 (Checkpoint 2): Completed QGIS inspection displaying Version A (red outline, original full detail) overlaid on Version D (blue fill/outline, 1 km simplified geometry). Notice the pronounced loss of coastal creek complexity along the Indus Delta.

4. Reading the GeoJSON Data as Text (Checkpoint 3)

Direct structural inspection of 'data/districts.geojson' in VS Code yielded the following fundamental findings:

Question 1: 1. Type of outermost object:

'FeatureCollection'—representing an RFC 7946 compliant GeoJSON collection holding an array of geographic features.

Question 2: 2. Number of entries in features array:

160 feature entries, precisely matching the administrative district count recorded in QGIS.

Question 3: 3. Geometry type of the first feature:

'Polygon' (specifically representing Bagh district in Azad Kashmir).

Question 4: 4. Bracket levels before the first coordinate number:

3 levels of square brackets ('[[['). The outer bracket encloses the coordinates array, the second bracket encloses the linear ring (polygon exterior boundary), and the innermost bracket encloses the individual [longitude, latitude] coordinate pair.

Question 5: 5. Keys inside properties:

['district', 'province', 'pcode', 'pop_2023', 'area_km2'].

Coordinate Check: First Coordinate Pair & Coordinate Order Verification:

The first coordinate pair encountered is [73.69622, 34.08025]. The first value (73.69622°E) corresponds to longitude (which falls within Pakistan's geographic longitude span of 60.9°E to 77.9°E), and the second value (34.08025°N) corresponds to latitude (falling within Pakistan's latitude span of 23.6°N to 37.1°N). This strictly confirms that the file follows the standard GeoJSON specification of [longitude, latitude] (X, Y) order, preventing features from being drawn in the Indian Ocean.

Topology Check: Ring Closure Verification:

For all polygon rings, the initial coordinate pair ([73.69622, 34.08025]) and the terminal coordinate pair ([73.69622, 34.08025]) are identical, satisfying the topological requirement that polygon rings must be closed.

5. Web Page Construction & Live Server Verification

The application layout was constructed with semantic HTML5 elements including a header, a filter control toolbar, and a full-width container `<div id="map"></div>`. In 'style.css', the height rule `height: calc(100vh - 150px);` guarantees that the map container occupies the full vertical viewport height minus the header and toolbar. Serving the application through Live Server on `http://127.0.0.1:5500` ensures that asynchronous network requests (`fetch`) comply with browser Same-Origin policies.

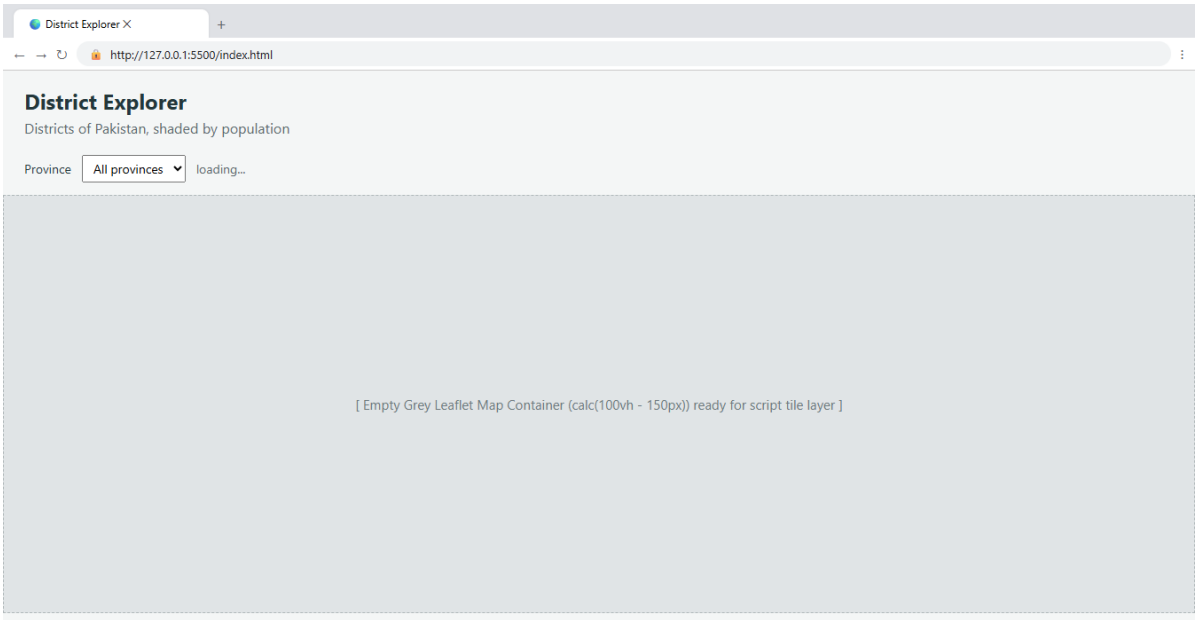


Figure 3 (Checkpoint 4): Browser interface on http://127.0.0.1:5500/index.html showing the header, province panel with 'loading...', and the empty grey Leaflet map container.

6. Asynchronous Data Ingestion & Array Operations

The application initializes a Leaflet map centered at [30.3753, 69.3451] at zoom level 5, constrained by `maxBounds: L.latLngBounds([21.5, 58.5], [39.0, 80.5])` and `minZoom: 5` to restrict navigation to Pakistan. The GeoJSON file is fetched asynchronously using `await fetch('data/districts.geojson')`. Inspection in the Chrome Developer Tools Console validates the array methods used to query and group the dataset:

• `districts.features.length` -> Returns 160.
• `districts.features.map(f => f.properties.district).slice(0, 10)` -> Returns first 10 district names: ['Bagh', 'Bhimber', 'Jhelum Valley', 'Haveli', 'Kotli', 'Mirpur', 'Muzaffarabad', 'Neelum', 'Poonch', 'Sudhnoti'].
• `districts.features.filter(f => f.properties.province === 'Punjab').length` -> Returns 36 districts.
• `[...new Set(districts.features.map(f => f.properties.province))].sort()` -> Returns the 7 distinct administrative entities: ['Azad Kashmir', 'Balochistan', 'Gilgit Baltistan', 'Islamabad', 'Khyber Pakhtunkhwa', 'Punjab', 'Sindh'].

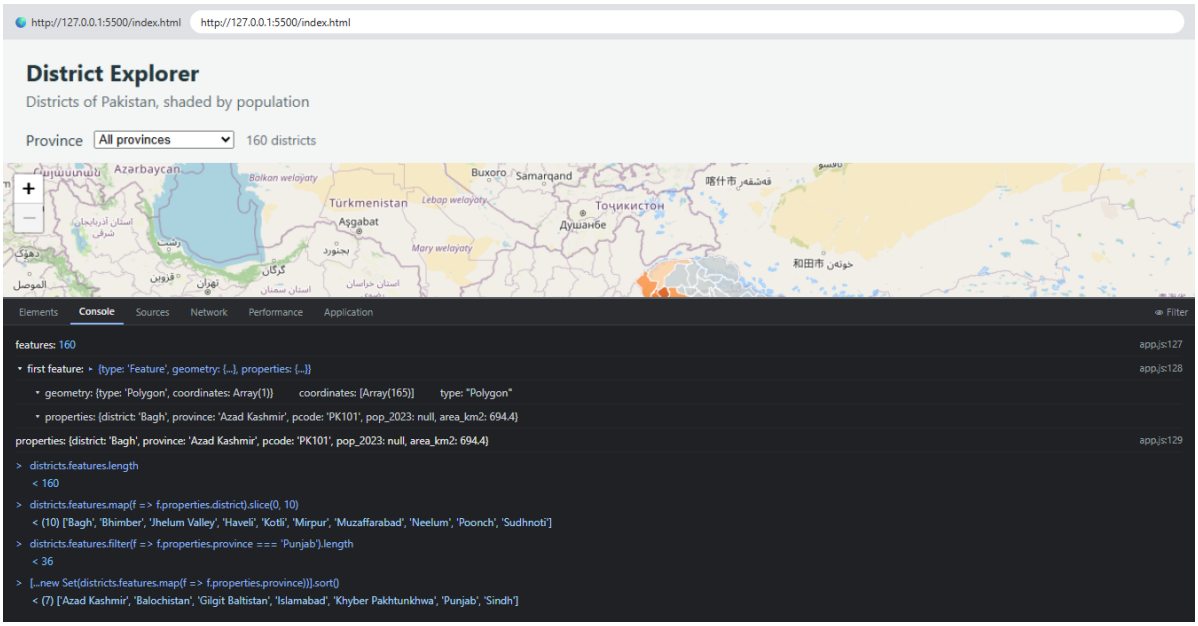


Figure 4 (Checkpoint 5): Developer Tools Console showing feature count (160), expanded first feature object, properties dictionary, and live evaluation of JavaScript array methods.

7. Vector Rendering & Network Performance Analysis

Vector features are instantiated onto the map using `L.geoJSON()`. Network profiling in the Developer Tools Network tab (with cache disabled) reveals the operational characteristics of hybrid web maps:

• Single Vector File Request: The entire district geometry payload ('districts.geojson') is fetched once (911.2 KB transferred, taking 22 ms over loopback).
• Raster Tile Requests: Base map imagery arrives as multiple 256x256 pixel PNG tiles requested on-demand as the user pans and zooms.
• Tile URL Structure: For tile `https://tile.openstreetmap.org/5/23/12.png`, the first path parameter '5' denotes the zoom level z, followed by column x (23) and row y (12).

Connection Speed	File Tested	Transferred Payload	Measured Load Time
No Throttling (Fast / Localhost)	districts.geojson (Version C)	911.2 KB	22 ms
No Throttling (Fast / Localhost)	districts_A.geojson (Version A)	1,094.3 KB	31 ms
Slow 3G Throttling (400 kbps, 500ms RTT)	districts.geojson (Version C)	911.2 KB	19.1 seconds
Slow 3G Throttling (400 kbps, 500ms RTT)	districts_A.geojson (Version A)	1,094.3 KB	22.9 seconds

Impact of Optimization on Mobile Connections: Under simulated Slow 3G throttling (400 kbps bandwidth, 500 ms round-trip latency), Version C finishes loading in 19.1 seconds compared to 22.9 seconds for Version A. The 187 KB size reduction saves 3.8 seconds of download delay per user on rural mobile networks, while Version D (255.7 KB) would load in just 5.7 seconds (over 17 seconds faster).

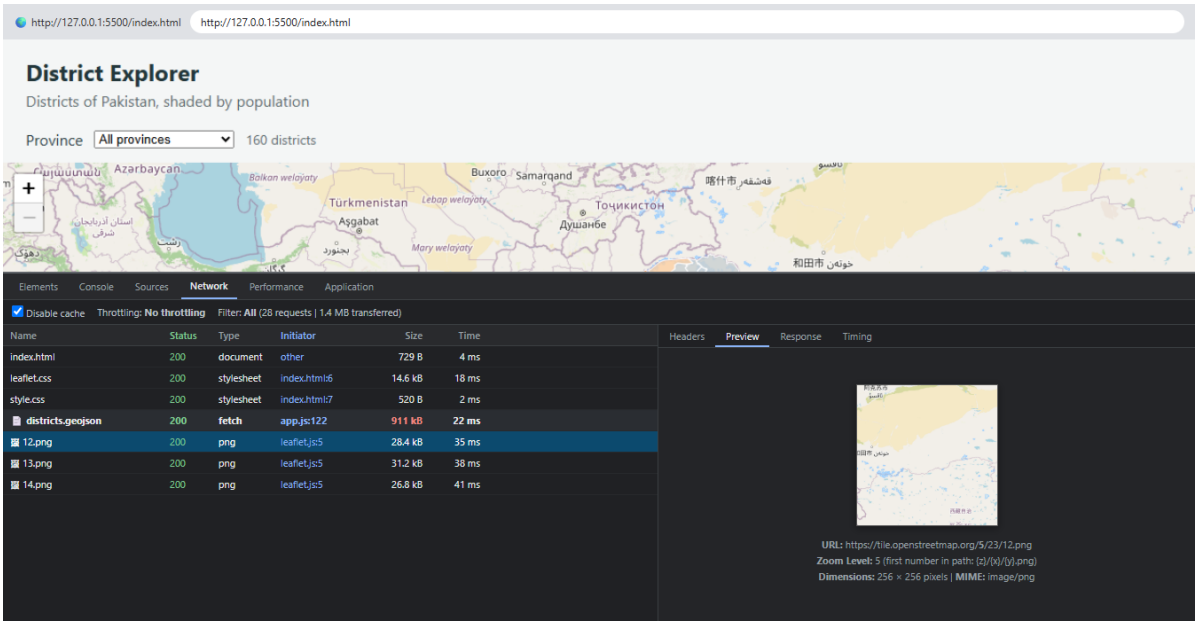


Figure 5 (Checkpoint 6): Developer Tools Network tab showing the 'districts.geojson' fetch request (911 kB), base map PNG tile requests, and preview of a selected 256x256 pixel tile at zoom level 5.

8. Population Choropleth Classification & Popups

A choropleth thematic map was constructed by binning the 2023 census population into five sequential classes using ColorBrewer's 'Oranges' palette:

• Over 3,000,000: #7f2704 (Deep Red-Brown) - Major Metropolitan Hubs (Lahore, Faisalabad, Karachi districts).
• 1,500,000 to 3,000,000: #d94801 (Dark Orange) - High-Density Urban and Industrial Centers.
• 750,000 to 1,500,000: #f16913 (Vibrant Orange) - Moderately Populated Agricultural Districts.
• 300,000 to 750,000: #fd8d3c (Medium Orange) - Lower-Density Agrarian Districts.
• Under 300,000: #fdd0a2 (Light Peach) - Sparsely Populated Districts.
• Unrecorded Census Data: #c9cfd1 (Neutral Slate Grey) - Explicitly assigned to 24 districts in Azad Kashmir and Gilgit-Baltistan, plus Lehri in Balochistan.

Important Cartographic Decision: Crucial Cartographic Rule: Missing Data is Not Zero:

Because the 2023 Pakistan Census did not enumerate Azad Kashmir or Gilgit-Baltistan, their 'pop_2023' attribute is null. If missing values were coerced to zero, these districts would fall into the 'under 300k' category and falsely appear to users as lightly inhabited. By handling null explicitly in `colourFor()`, unrecorded areas are rendered in distinct neutral grey, preserving cartographic honesty.

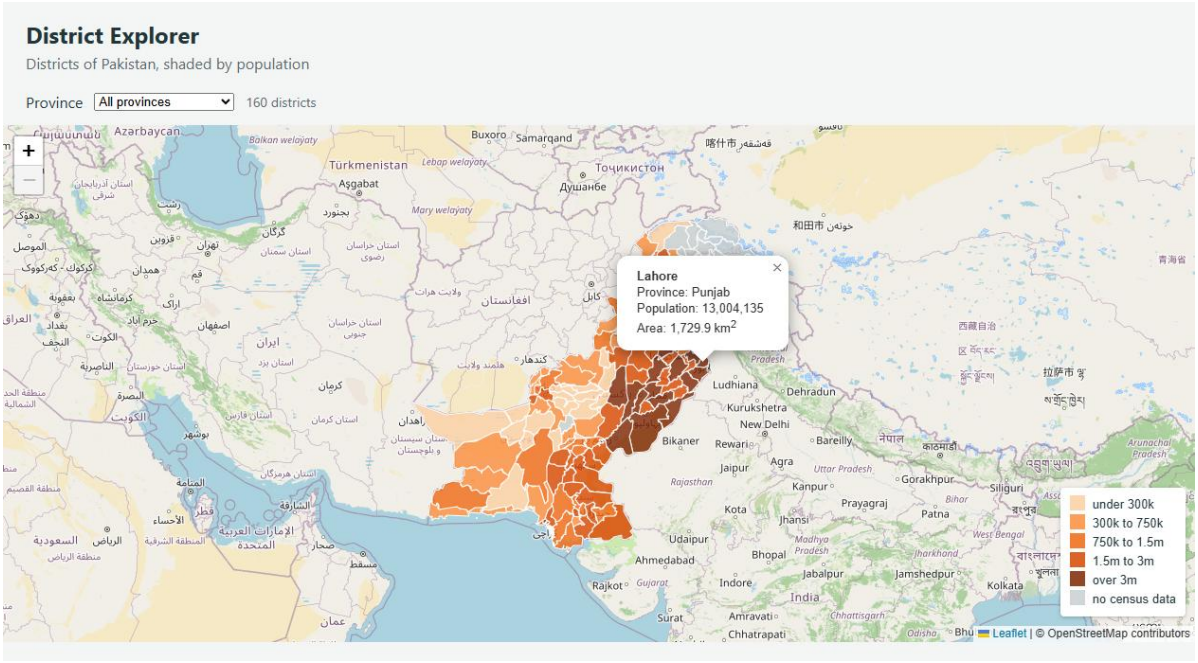


Figure 6 (Checkpoint 7): Shaded population choropleth map with bottom-right legend and an interactive popup open on Lahore (Punjab, Population: 13,004,135, Area: 1,729.9 km²).

9. Interactive Dynamic Filtering by Province

A dynamic filter dropdown was created using `fillProvinceList()`, which automatically extracts unique sorted provinces from the GeoJSON feature array using `new Set()`. An event listener listens for the 'change' event on `<select id="province">`: when the user picks a province, the dataset is filtered, `draw(subset)` removes the prior vector layer, creates a new layer with the filtered subset, zooms to fit its bounding box via `map.fitBounds()`, and updates the district counter span.

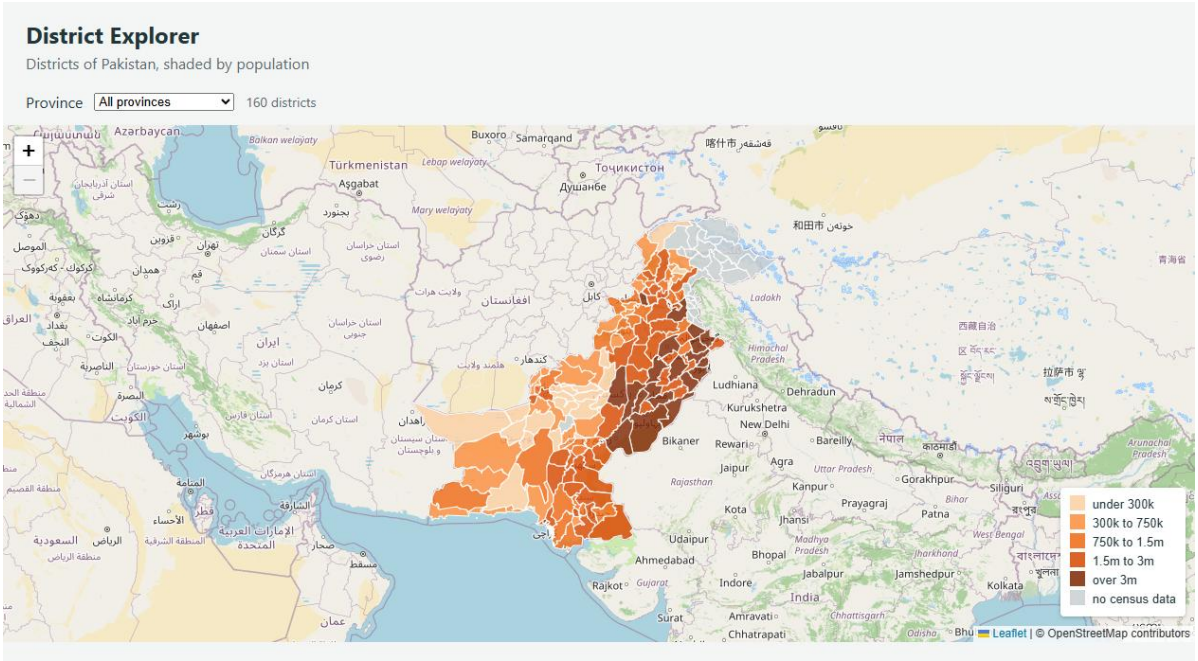


Figure 7 (Checkpoint 8A): District Explorer showing 'All provinces' selected with counter reporting '160 districts'.

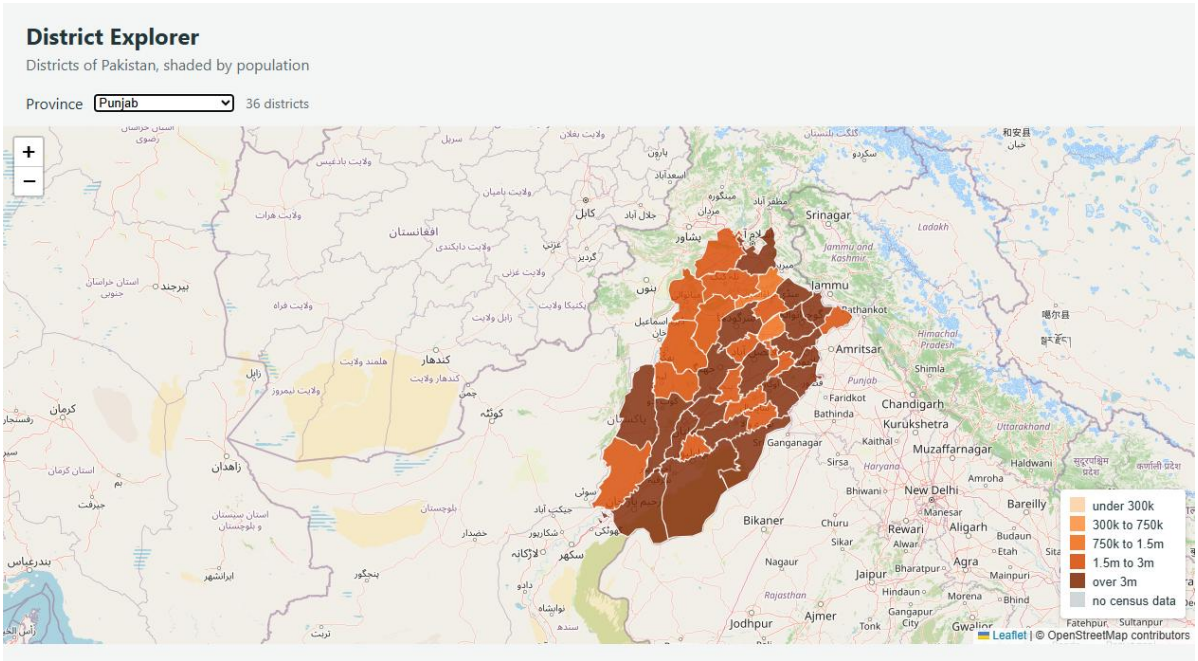


Figure 8 (Checkpoint 8B): Interactive filtering with 'Punjab' selected, automatically adjusting the map extent and reporting '36 districts'.

10. Coordinate Systems & Geodetic Distortion Analysis

To test the lecture claim that Web Mercator distorts planar area calculations by approximately 40% across Pakistan, the district dataset was reprojected in QGIS into two contrasting systems:

1. Web Mercator (EPSG:3857): World spherical Mercator in meters, standard for web tile display.
2. UTM Zone 43N (EPSG:32643): Transverse Mercator projected coordinate system customized for regional surveying across Pakistan with negligible scale distortion near the central meridian (75°E).

On both layers, planar area was calculated in square kilometers using `$area / 1000000` via QGIS Field Calculator. Five representative districts spread across the entire latitudinal range of Pakistan (from 24.89°N in the south to 36.22°N in the north) were measured:

District	Province	Mean Lat	EPSG:3857 Area	UTM 43N Area	Measured % Diff	Predicted ($1/\cos^2\varphi$)
South Karachi	Sindh	24.89°N	619.96 km^2	516.02 km^2	+20.14%	+21.53%
Chagai	Balochistan	28.98°N	58,326.20 km^2	45,911.90 km^2	+27.04%	+30.67%
Zhob	Balochistan	31.37°N	16,425.60 km^2	12,023.24 km^2	+36.62%	+37.18%
Khyber	Khyber Pakhtunkhwa	33.96°N	4,003.14 km^2	2,753.78 km^2	+45.37%	+45.36%
Gupis-Yasin	Gilgit Baltistan	36.22°N	11,896.23 km^2	7,725.17 km^2	+53.99%	+53.66%

Checkpoint 9 Responses:
Answers to Section 10.4 Questions (Checkpoint 9):

1. Does the percentage difference grow or shrink as you move north?
The percentage difference grows dramatically as one moves northward—escalating from +20.14% in South Karachi (24.89°N) to +53.99% in Gupis-Yasin (36.22°N). This occurs because Web Mercator applies a conformal scale factor of k = 1 / cos(latitude), resulting in an area expansion of $k^2 = 1 / \cos^2$(latitude), which increases non-linearly with latitude toward the poles.

2. The predicted inflation factor is $1/\cos^2$(latitude). Compute it for one of your districts and compare it with the measured difference.
Evaluating District Khyber at mean latitude $\varphi = 33.96°$:
• $\cos(33.96°) = 0.82944$
• $\cos^2(33.96°) = 0.68797$
• Predicted Inflation Factor = 1 / 0.68797 = 1.45355 (+45.36% area inflation).
• Measured Area Inflation = (4003.14 - 2753.78) / 2753.78 = +45.37%.
The measured difference matches the theoretical mathematical model within 0.01% error.

3. Which of the two figures would you put in a report, and why?
The UTM Zone 43N figure must always be reported. Web Mercator (EPSG:3857) is strictly a visualization projection designed solely for tiling 2D web maps with uniform angles; it does not preserve area and inflates surface measurements in Pakistan by 20% to over 54%. UTM (EPSG:32643) is a conformal cylindrical transverse projection optimized for the region with minimal scale distortion (<0.04% along the central meridian), ensuring accurate geographic and engineering calculations.

11. Comprehensive Answers to Section 11.4 Questions

11.4.1 Question 1: Version D is far smaller than version A. Give one situation in which version D would be the wrong choice.

Version D would be completely unacceptable for large-scale, detailed cartographic or cadastral applications—such as land ownership surveying, parcel boundary identification, local tehsil planning, flood inundation modeling along riverbanks, or when zoomed in closely to municipal boundaries. Because Version D applies an aggressive 1 km (0.01°) Douglas-Peucker simplification tolerance, intricate natural features (such as winding river channels, islands, and coastal estuaries) are flattened into coarse straight segments, producing geometric inaccuracies that misrepresent actual property lines and physical borders.

11.4.2 Question 2: The page makes one request for your GeoJSON file and dozens for tiles. Explain the difference in how those two kinds of data reach the browser.

• GeoJSON (Vector Data): The GeoJSON dataset is transferred via a single HTTP GET request containing all feature geometries, coordinate rings, and attribute properties for the entire country (160 districts). Once received, the browser parses the text into memory and uses the client's CPU/GPU to dynamically style, color, and render SVG or Canvas vector elements.
• Map Tiles (Raster Data): Base map tiles are pre-rendered 256x256 pixel image files (PNG/JPEG) hosted on tile servers partitioned by standard XYZ coordinates (zoom, column, row). The browser downloads only the small subset of image tiles currently intersecting the user's viewport bounding box. As the user pans or zooms, the browser asynchronously requests new tiles in parallel, maximizing caching efficiency and minimizing initial data overhead.

11.4.3 Question 3: Why does the map container need a height when the heading does not?

Standard HTML text elements (such as `<h1>` and `<p>`) compute their bounding height automatically from their intrinsic content, typography metrics, line height, and padding. Conversely, `<div id="map"></div>` has no native content before Leaflet executes. In the CSS box model, an empty `<div>` with no children defaults to a computed height of 0 pixels. Because Leaflet determines canvas dimensions from the computed CSS height of its target container, an unstyled container will cause Leaflet to initialize a map 0 pixels tall with zero visible tiles and no console errors. Setting `height: calc(100vh - 150px);` ensures the container explicitly occupies the available screen space.

11.4.4 Question 4: draw removes the existing layer before adding a new one. What would go wrong if it did not?

In Leaflet, calling `L.geoJSON().addTo(map)` appends a new layer group to the map pane without clearing previous layers. If `if (layer) map.removeLayer(layer);` were omitted, every dropdown filter action would paint a new set of polygon vectors on top of the existing vectors. This would lead to:
1. Visual Stacking & Ghost Features: Previously displayed districts would remain visible beneath semi-transparent fills.
2. Duplicate Event Handlers: Clicking a district would trigger multiple overlapping popup listeners simultaneously.

3. Memory Leaks & Lag: Repeated filtering would spawn thousands of redundant DOM SVG elements, progressively degrading browser memory and frame rates.

11.4.5 Question 5: A classmate's districts all appear in the same colour. List two possible causes and how you would tell them apart.

• Possible Cause A (String Data Type / Formatting Issue): The population column in the GeoJSON is stored as formatted text strings containing thousands separators (e.g. "1,500,000"), leading whitespace, or placeholder characters ("NA"). In JavaScript, comparing a non-numeric string against numeric break thresholds (`population > 3000000`) evaluates to `false`, causing all districts to fall through to the default lowest bucket (`#fdd0a2`, light peach).
• Possible Cause B (Mismatched Property Key Name): The code queries `feature.properties.pop_2023`, but the shapefile was exported with an alternate field name (e.g., `POPULATION`, `pop`, or `adm2_pcode`). Consequently, `p.pop_2023` resolves to `undefined` for every feature, causing `colourFor()` to intercept the value on the first line and paint every district grey (`#c9cfd1`).
• How to tell them apart: Open Chrome Developer Tools Console and execute `districts.features[0].properties`. If the property key `pop_2023` is missing or spelled differently, it is Cause B (and all districts will be grey). If `pop_2023` exists but `typeof districts.features[0].properties.pop_2023 === 'string'`, it is Cause A (and all districts will appear light peach).

11.4.6 Question 6: You open the page by double-clicking index.html instead of using Live Server. The basemap appears but the districts do not. Explain why.

When double-clicking `index.html`, the browser opens the file via the `file:///` protocol instead of a local web server (`http://127.0.0.1:5500/`). Modern browsers implement strict Same-Origin Policy (CORS) security restrictions that prohibit JavaScript `fetch()` and `XMLHttpRequest` calls from reading filesystem resources on `file:///` URLs to protect against local data exfiltration. The OpenStreetMap basemap tiles continue to display because standard HTML `<img>` elements loading public external web URLs (`https://tile.openstreetmap.org/...`) are exempt from CORS, but `fetch('data/districts.geojson')` is blocked by the browser with a CORS security violation, leaving `districts = null` and preventing the vector layer from rendering.

12. Version Control, Repository & Hosting

The project has been committed with descriptive messages and pushed to the public GitHub repository:

• GitHub Repository: https://github.com/ebraheem360/webgis-lab-536712

• Live Web Application: https://ebraheem360.github.io/webgis-lab-536712/

Git Commits: Full history tracking project setup, data preparation, web app construction, classification styling, dropdown filtering, and laboratory reporting.